

GTOS Handbook - Foundations, Canon, Constitution, Vision, and Business Model

GTOS Enterprise Engineering Handbook - Final Edition 1.0

Source Chapters

- Causality, Contradiction, and Correction
- Deontic and Authority Model
- Epistemology of Trade
- GTOS Reference Theory
- GTOS Meta-Model
- Ontology of Trade
- Uncertainty Model
- CANON-009 — Knowledge Is Not Reality
- CANON-010 — Decisions Transform Knowledge into Commitment
- CANON-011 — Intent Precedes Execution
- CANON-001 — Reality Precedes Representation
- CANON-002 — Truth Is Singular; Perspectives Are Plural
- Volume 00 — The GTOS Canon
- CANON-003 — Identity Precedes Relationship
- CANON-004 — Time Is a First-Class Business Dimension
- CANON-005 — State Is the Observable Condition of Reality
- CANON-006 — Relationships Are Independent Business Facts
- CANON-007 — Events Describe State Transitions
- CANON-008 — History Preserves Truth
- Constitutional Amendment Process
- Article 01 — Supremacy and Scope
- Article 02 — Truth Ownership
- Article 03 — Identity and Authority
- Article 04 — Decision Governance

- Article 05 — Historical Integrity
- Article 06 — Interoperability
- Article 07 — AI Accountability
- Article 08 — Security and Privacy
- Article 09 — Conformance and Exceptions
- Article 10 — Evolution and Amendment
- Volume 01 — GTOS Constitution
- Volume 02 — Vision
- Strategic Principles
- Target Operating Model
- Accountability and Decision Rights
- Capability 01 — Party and Identity
- Capability 02 — Product and Classification
- Capability 03 — Commercial Agreement
- Capability 04 — Order and Fulfilment
- Capability 05 — Shipment and Transport
- Capability 06 — Trade Documentation
- Capability 07 — Compliance and Customs
- Capability 08 — Finance and Insurance
- Capability 09 — Risk and Exception
- Capability 10 — Decision Intelligence
- Capability 11 — Audit and Evidence
- Volume 03 — Business Architecture
- Value Streams
- Business Case and Value Model
- Executive Architecture Brief
- Executive Risk Register
- Glossary

Causality, Contradiction, and Correction

Causality

GTOS distinguishes temporal sequence from causality. An Event occurring before another Event does not prove causation. Causal claims SHALL identify supporting evidence and assumptions.

Correlation and Causation Identifiers

- correlationId groups related work.
- causationId identifies the directly initiating fact or request.
- Neither field alone proves legal or scientific causality.

Correction

A correction creates a new governed assertion that qualifies, supersedes, or invalidates a prior assertion. The original assertion remains historically visible.

Retraction

Retraction states that an assertion should no longer be relied upon. Retraction does not claim the assertion never existed.

Contradiction Resolution

Resolution SHALL identify:

- contradictory claims;
- applicable authority;
- evidence considered;
- decision time;
- resulting authoritative claim;
- effect on downstream decisions;
- whether remediation is required.

Deontic and Authority Model

Deontic Concepts

GTOS distinguishes:

- **Permission:** an action may be performed.
- **Obligation:** an action is required.
- **Prohibition:** an action must not be performed.
- **Authority:** an actor may make a binding assertion or decision.
- **Delegation:** bounded authority is granted to another actor.
- **Mandate:** an actor is instructed to pursue an objective within constraints.
- **Liability:** accountability for a consequence or failure.

Authority Invariants

1. Technical capability does not imply business authority.
2. Access to data does not imply authority to alter truth.
3. AI recommendation does not imply authority to decide.
4. Authority SHALL have scope, source, validity, and revocation semantics.
5. Delegation SHALL NOT silently expand through service composition.
6. Emergency authority SHALL expire and be reviewed.

Policy Result

A policy evaluation may return permit, deny, require approval, require evidence, or require additional obligations. It SHALL identify the policy version and evaluation context.

Epistemology of Trade

Purpose

Epistemology governs what GTOS knows, why it believes it, who asserted it, when it became available, and how uncertainty or contradiction is preserved.

Observation

An Observation is an observer's assertion about reality. Sensors, humans, organizations, external systems, documents, and AI models may produce observations.

Claim

A Claim is a proposition asserted to be true. Claims may be authoritative, corroborated, disputed, inferred, or rejected.

Evidence

Evidence supports or challenges a Claim. Evidence has origin, integrity, scope, time, accessibility, and relevance.

Confidence

Confidence expresses support for a Claim; it is not truth. Confidence SHALL NOT be used to disguise the absence of authority.

Contradiction

Contradiction is a first-class condition in which incompatible Claims coexist. GTOS preserves contradictions until an authorized resolution occurs.

Knowledge

Knowledge is the set of Claims available to an observer or decision process at a time, together with provenance and qualification.

Decision

A Decision is a governed commitment made using an Evidence Set, applicable policy, objectives, and authority.

Hindsight Discipline

Decision review SHALL reconstruct the knowledge available at decision time. Later discoveries may affect learning, but SHALL NOT be presented as if they were previously available.

GTOS Reference Theory

The GTOS Reference Theory explains what GTOS assumes about trade reality, knowledge, authority, decision, and action.

Four Planes

Ontological Plane

Defines what exists: entities, identities, time, states, relationships, events, and history.

Epistemological Plane

Defines what is known: observations, claims, evidence, confidence, contradictions, knowledge, and decisions.

Deontic Plane

Defines what is permitted, required, prohibited, delegated, or obligated.

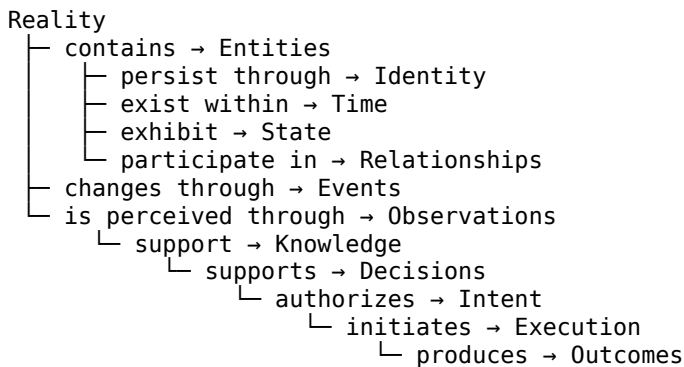
Operational Plane

Defines intent, command, execution, outcome, correction, and recovery.

The planes are distinct but connected. An implementation that collapses them may appear simpler while producing incorrect audit, security, AI, or business behavior.

GTOS Meta-Model

Dependency Chain



History preserves Events, Observations, Decisions, Intent, Execution, Outcomes, and Corrections.

Authority constrains assertions, decisions, delegation, and execution.

Policy evaluates knowledge and context to produce permissions, obligations, or prohibitions.

Meta-Model Invariants

1. No Relationship exists without independently identifiable participants.
2. No Event is meaningful without a subject, occurrence time, and transition semantics.
3. No historical Decision may be evaluated using information unavailable at decision time.
4. No Execution may claim legitimate business purpose without traceable Intent.
5. No derived view becomes authoritative merely through replication.
6. No correction erases the existence of the assertion being corrected.
7. No AI capability possesses authority unless authority has been explicitly delegated.

Canon Admission Test

A candidate Canon is admissible only when it passes:

- **Independence:** the concept is not reducible to another Canon.
- **Necessity:** removing it creates incoherence or makes a required GTOS property impossible.
- **Derivation:** lower-level standards can be logically derived from it.
- **Placement:** it has a unique location in the Meta-Model.
- **Testability:** architectural consequences can be inspected.

Ontology of Trade

Purpose

Ontology defines the kinds of things GTOS can assert exist and the conditions under which those assertions remain coherent.

Entity

An Entity is anything that must remain distinguishable for business purposes. Parties, products, facilities, shipments, documents, accounts, contracts, licenses, containers, vehicles, and regulatory classifications may be Entities.

Entity is not synonymous with database row. A row is a representation. Identity continuity may survive merges, migrations, renaming, or system replacement.

Identity

Identity provides continuity through change. GTOS distinguishes:

- intrinsic identity, where a thing is considered the same thing;
- assigned identifiers, such as registration numbers;
- contextual identifiers, such as partner-specific codes;
- credentials, which demonstrate control but do not define existence.

Time

GTOS uses multiple temporal dimensions:

- occurrence time;
- effective or validity time;
- observation time;
- recording time;
- processing time;
- correction time.

State

State is a scoped set of propositions about a subject. State may be recorded, derived, projected, or disputed. The state owner and derivation method SHALL be known.

Relationship

A Relationship is an independent business fact among identified participants. Contracts, qualifications, mandates, delegations, ownership, custody, and service agreements are not mere foreign keys.

Event

An Event is an immutable assertion that something occurred. Events describe transitions or significant facts; they do not command future work.

History

History is the preserved sequence through which past state and past knowledge can be reconstructed.

Uncertainty Model

Forms of Uncertainty

GTOS recognizes:

- missing information;
- measurement error;
- ambiguous identity;
- model uncertainty;
- future uncertainty;
- policy ambiguity;
- disputed authority;
- contradictory evidence.

Rules

- Unknown SHALL NOT be encoded as false.
- Estimated SHALL NOT be encoded as observed.
- Predicted SHALL NOT be encoded as decided.
- Confidence SHALL include method and calibration context where material.
- Decision thresholds SHALL be policy-controlled.
- High-impact uncertainty SHALL trigger escalation or evidence acquisition.

AI Consequence

An AI agent must be able to say that the available evidence is insufficient. Fabricated certainty is a conformance failure.

CANON-009 — Knowledge Is Not Reality

Canon Statement

Reality exists independently of what any observer knows. GTOS SHALL model Observations, Claims, Knowledge, provenance, and confidence separately from reality.

Classification

Property	Value
Canon ID	CANON-009
Plane	Epistemology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-001, CANON-004, CANON-008

Normative Statements

- Reality exists independently of what any observer knows. GTOS SHALL model Observations, Claims, Knowledge, provenance, and confidence separately from reality.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Platforms treat database contents as complete reality, causing delayed, missing, contradictory, and uncertain information to be mishandled.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

Reality exists independently of what any observer knows. GTOS SHALL model Observations, Claims, Knowledge, provenance, and confidence separately from reality.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Observation

An observer’s assertion about reality.

Claim

A proposition asserted to be true.

Knowledge Set

Claims available to an observer or process at a time.

Confidence

Qualified support for a Claim, not authority.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A refrigeration failure occurs at 14:02, a sensor observes it at 14:05, GTOS records it at 14:09, and an operator learns of it at 14:10. The failure occurred once; knowledge propagated over time.

Failure Modes and Risks

- Hindsight bias contaminates decision evaluation.
- AI invents certainty.
- Contradictory observations are overwritten.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Are occurrence and observation time distinct?
- ☐ Is provenance recorded?
- ☐ Can decision-time knowledge be reconstructed?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-010 — Decisions Transform Knowledge into Commitment

Canon Statement

A Decision SHALL be a governed commitment made from an explicit Knowledge Set, applicable policy, objectives, and authority.

Classification

Property	Value
Canon ID	CANON-010
Plane	Epistemology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-002, CANON-008, CANON-009

Normative Statements

- A Decision SHALL be a governed commitment made from an explicit Knowledge Set, applicable policy, objectives, and authority.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Recommendations, model outputs, approvals, determinations, and system side effects are frequently conflated. The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

A Decision SHALL be a governed commitment made from an explicit Knowledge Set, applicable policy, objectives, and authority.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Decision

A binding selection or determination.

Recommendation

A non-binding proposal.

Decision Authority

The recognized right to decide.

Evidence Set

The claims and evidence considered.

Decision Record

The durable evidence of decision context and result.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

An AI may recommend inspection. The customs officer or delegated automated policy may decide inspection. The inspection workflow executes that decision.

Failure Modes and Risks

- AI output gains hidden authority.
- Approvals cannot be audited.
- Policies cannot be reproduced.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Is decision authority explicit?
- ☐ Are evidence and policy versions captured?
- ☐ Is recommendation distinct from decision?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-011 — Intent Precedes Execution

Canon Statement

Legitimate Execution SHALL be traceable to explicit, authorized Intent; execution capability SHALL NOT invent business purpose.

Classification

Property	Value
Canon ID	CANON-011
Plane	Operational Boundary
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-003, CANON-004, CANON-010

Normative Statements

- Legitimate Execution SHALL be traceable to explicit, authorized Intent; execution capability SHALL NOT invent business purpose.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Automation often acts because it technically can, without a durable statement of objective, authority, constraints, or expected outcome.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

Legitimate Execution SHALL be traceable to explicit, authorized Intent; execution capability SHALL NOT invent business purpose.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Intent

An authorized expression of desired future effect.

Execution

An attempt to realize Intent.

Outcome

The observed result of Execution.

Mandate

A bounded authorization to pursue a class of Intent.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A payment service does not decide to pay because an invoice exists. An authorized Intent to settle an obligation initiates payment execution under policy and limits.

Failure Modes and Risks

- Agents exceed delegated purpose.
- Technical retries create unauthorized duplicate effects.
- Failures cannot be related to business objectives.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Can every execution be traced to Intent?
- ☐ Does Intent include authority and constraints?
- ☐ Are outcome and failure recorded independently?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-001 — Reality Precedes Representation

Canon Statement

Trade reality exists independently of GTOS. GTOS SHALL represent reality and SHALL NOT redefine reality merely to simplify software.

Classification

Property	Value
Canon ID	CANON-001
Plane	Foundation
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	None

Normative Statements

- Trade reality exists independently of GTOS. GTOS SHALL represent reality and SHALL NOT redefine reality merely to simplify software.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Enterprise platforms routinely convert technical constraints into supposed business truths. Required fields, fixed work-flows, and database ownership are then mistaken for properties of trade itself.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

Trade reality exists independently of GTOS. GTOS SHALL represent reality and SHALL NOT redefine reality merely to simplify software.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Reality

The trade world that exists independently of GTOS.

Representation

A model, record, message, document, view, or computation describing reality.

Model Error

A divergence between a representation and verified business reality.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A port may accept a cargo movement before a partner platform has created its local shipment record. The cargo movement is real; the absence of a record is a knowledge and integration problem.

Failure Modes and Risks

- Software workflow becomes a hidden trade policy.
- Migration changes historical meaning.
- AI learns implementation artifacts as if they were trade laws.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Can the domain expert reject the model without being told the business is wrong?
- ☐ Can the implementation be replaced without changing the represented business concept?
- ☐ Are missing records distinguished from non-existent business facts?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.
- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-002 — Truth Is Singular; Perspectives Are Plural

Canon Statement

Within each governed proposition and authority boundary, GTOS SHALL identify one authoritative truth while preserving multiple contextual perspectives.

Classification

Property	Value
Canon ID	CANON-002
Plane	Foundation
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-001

Normative Statements

- Within each governed proposition and authority boundary, GTOS SHALL identify one authoritative truth while preserving multiple contextual perspectives.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Distributed organizations often create duplicate masters. Reconciliation then becomes a competition between databases rather than a governed resolution of authority.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

Within each governed proposition and authority boundary, GTOS SHALL identify one authoritative truth while preserving multiple contextual perspectives.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Authoritative Truth

The claim accepted by the responsible authority within a defined scope.

Perspective

A contextual projection or interpretation of authoritative truth.

Truth Owner

The accountable authority governing a proposition.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

The customs authority owns the legal clearance determination. A carrier may hold a useful operational view of clearance, but that view does not become the legal authority.

Failure Modes and Risks

- Conflicting records trigger inconsistent execution.
- Replicas silently become masters.
- Analytics combine incompatible definitions.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Is the truth owner explicit for each governed proposition?
- ☐ Are projections labeled as perspectives?
- ☐ Can disagreement be represented without destructive overwrite?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.
- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

Volume 00 — The GTOS Canon

Within GTOS, **Canon** means the set of foundational axioms from which constitutional rules, reference architectures, engineering standards, and conformance tests are derived.

The Canon is intentionally small. Good ideas that are derivable, domain-specific, or technology-specific belong downstream.

Canon Sequence

ID	Canon	Plane
001	Reality Precedes Representation	Foundation
002	Truth Is Singular; Perspectives Are Plural	Foundation
003	Identity Precedes Relationship	Ontology
004	Time Is a First-Class Business Dimension	Ontology
005	State Is the Observable Condition of Reality	Ontology
006	Relationships Are Independent Business Facts	Ontology
007	Events Describe State Transitions	Ontology
008	History Preserves Truth	Ontology
009	Knowledge Is Not Reality	Epistemology
010	Decisions Transform Knowledge into Commitment	Epistemology
011	Intent Precedes Execution	Operational Boundary

CANON-003 — Identity Precedes Relationship

Canon Statement

An Entity SHALL be independently identifiable before GTOS establishes a Relationship involving that Entity.

Classification

Property	Value
Canon ID	CANON-003
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-001

Normative Statements

- An Entity SHALL be independently identifiable before GTOS establishes a Relationship involving that Entity.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Systems frequently identify parties or assets only through their current relationship, causing identity loss when ownership, contracts, or roles change.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

An Entity SHALL be independently identifiable before GTOS establishes a Relationship involving that Entity.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Entity

A distinguishable participant, object, place, instrument, document, or concept.

Identity

Continuity through which an Entity remains distinguishable across change.

Identifier

A governed symbol referring to an Identity.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A freight forwarder remains the same legal party after changing carriers, offices, or customer relationships. Those changes affect relationships and attributes, not foundational identity.

Failure Modes and Risks

- Duplicate parties evade controls.
- Relationship termination appears to delete an Entity.
- Audit chains break across identifier changes.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Can the Entity be referenced independently of any current Relationship?
- ☐ Are identifier scope and issuer recorded?
- ☐ Can merges and splits preserve history?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.
- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-004 — Time Is a First-Class Business Dimension

Canon Statement

GTOS SHALL model time explicitly and SHALL distinguish occurrence, validity, observation, recording, processing, and correction time where material.

Classification

Property	Value
Canon ID	CANON-004
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-001

Normative Statements

- GTOS SHALL model time explicitly and SHALL distinguish occurrence, validity, observation, recording, processing, and correction time where material.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Single timestamps cannot answer what happened, what was effective, what was known, or when the platform processed it.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

GTOS SHALL model time explicitly and SHALL distinguish occurrence, validity, observation, recording, processing, and correction time where material.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Occurrence Time

When something happened in reality.

Validity Time

When a proposition was effective.

Observation Time

When an observer became aware.

Recording Time

When GTOS persisted the assertion.

Processing Time

When a process handled it.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A certificate may be issued on Monday, become valid Wednesday, be received Friday, and be corrected the following month. Each time answers a different question.

Failure Modes and Risks

- Historical compliance is evaluated incorrectly.
- Late data changes past analytics silently.
- Decision review suffers hindsight bias.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Are temporal meanings named rather than implied?
- ☐ Can past valid state be reconstructed?
- ☐ Are late and corrected observations preserved?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-005 — State Is the Observable Condition of Reality

Canon Statement

State SHALL describe the scoped condition of an identified subject at a defined time and SHALL distinguish recorded, derived, projected, and disputed state.

Classification

Property	Value
Canon ID	CANON-005
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-001, CANON-003, CANON-004

Normative Statements

- State SHALL describe the scoped condition of an identified subject at a defined time and SHALL distinguish recorded, derived, projected, and disputed state.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Status fields compress complex business conditions into ambiguous labels and often conceal who owns the state or how it was derived.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

State SHALL describe the scoped condition of an identified subject at a defined time and SHALL distinguish recorded, derived, projected, and disputed state.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

State

A set of propositions describing a subject under scope and time.

Invariant

A condition that must hold across valid transitions.

Derived State

State computed from facts and rules.

Projected State

A purpose-specific view of state.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A shipment can be physically at a terminal, commercially on hold, customs-cleared, financially unpaid, and operationally delayed at the same moment. These are scoped states, not one universal status.

Failure Modes and Risks

- One status field controls unrelated decisions.
- Derived state is mistaken for authority.
- Invalid transitions bypass invariants.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Is state scoped by domain and owner?
- ☐ Are invariants testable?
- ☐ Can derived state be recomputed from evidence?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-006 — Relationships Are Independent Business Facts

Canon Statement

A business-significant Relationship SHALL be modeled as an independent fact with roles, attributes, lifecycle, validity, and history.

Classification

Property	Value
Canon ID	CANON-006
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-003, CANON-004, CANON-005

Normative Statements

- A business-significant Relationship SHALL be modeled as an independent fact with roles, attributes, lifecycle, validity, and history.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Foreign-key modeling hides contractual terms, delegation, custody, qualification, and obligations that belong to the relationship itself.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

A business-significant Relationship SHALL be modeled as an independent fact with roles, attributes, lifecycle, validity, and history.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Relationship

A meaningful association among independently identified Entities.

Role

Participation semantics within a Relationship.

Relationship State

The scoped condition of a Relationship.

Relationship Validity

The time during which the Relationship has effect.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

A supplier qualification links buyer and supplier but has its own audit evidence, scope, approval, expiry, suspension, and renewal history.

Failure Modes and Risks

- Contract terms are stored on the wrong party.
- Delegation cannot be revoked cleanly.
- Network risk cannot be analyzed.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Do relationship attributes belong to neither participant independently?
- ☐ Can multiple Relationships coexist between the same parties?
- ☐ Is relationship history preserved?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-007 — Events Describe State Transitions

Canon Statement

An Event SHALL assert that a business-significant occurrence has happened and SHALL NOT be used as a disguised Command.

Classification

Property	Value
Canon ID	CANON-007
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-004, CANON-005

Normative Statements

- An Event SHALL assert that a business-significant occurrence has happened and SHALL NOT be used as a disguised Command.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Message-driven systems often label requests as events, making failure, authority, and audit semantics ambiguous. The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

An Event SHALL assert that a business-significant occurrence has happened and SHALL NOT be used as a disguised Command.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

Event

An immutable assertion of a completed occurrence.

Command

A request to attempt an action.

Transition

A change from one valid State to another.

Outcome Event

An Event reporting the result of execution.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

ReleaseShipment is a Command. ShipmentReleased is an Event. ShipmentReleaseRejected is also an Event and preserves the attempted outcome.

Failure Modes and Risks

- Consumers assume work succeeded when it was merely requested.
- Retries duplicate business effects.
- Audit cannot distinguish request from fact.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Is the event named in past tense?
- ☐ Does it describe something already completed?
- ☐ Are subject, occurrence time, source, and schema version explicit?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.

- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

CANON-008 — History Preserves Truth

Canon Statement

GTOS SHALL preserve governed history and SHALL correct through new facts rather than rewriting the past.

Classification

Property	Value
Canon ID	CANON-008
Plane	Ontology
Priority	Absolute
Scope	Entire GTOS Ecosystem
Status	Normative
Depends On	CANON-002, CANON-004, CANON-007

Normative Statements

- GTOS SHALL preserve governed history and SHALL correct through new facts rather than rewriting the past.
- Implementations SHALL preserve the distinctions defined by this Canon across APIs, data, events, AI behavior, security policy, and operations.
- A derived view SHALL NOT claim greater authority than its governed source.
- Conformance evidence SHALL be inspectable and reproducible.

Historical Context

Mutable records optimize present-day convenience at the cost of legal, operational, and analytical truth.

The resulting failures are not merely data-quality issues. They create invalid authority, incorrect automation, misleading analytics, and brittle integration. This Canon establishes the conceptual boundary required to prevent those failures.

Core Philosophy

GTOS SHALL preserve governed history and SHALL correct through new facts rather than rewriting the past.

The Canon is independent of implementation style. A monolith, distributed platform, event-driven system, or agentic architecture may conform, but each must preserve the same semantic distinctions.

Formal Definitions

History

The ordered record of facts, observations, decisions, and corrections.

Correction

A new assertion qualifying or superseding a prior assertion.

Reconstruction

Deriving a past State or Knowledge Set from historically valid information.

Formal Invariants

1. The concept defined by this Canon SHALL have explicit ownership and scope.
2. Its identity and temporal meaning SHALL remain stable across representation changes.
3. Any derived assertion SHALL retain provenance to its inputs.
4. Corrections SHALL preserve historical evidence.
5. Authority SHALL NOT be inferred from technical possession or processing.

Engineering Interpretation

The Canon affects aggregate design, data contracts, event envelopes, authorization policy, observability, AI prompts and tools, audit records, and migration procedures.

Engineering teams SHALL encode the required distinctions in types and contracts rather than relying only on prose. Invalid states and invalid transitions SHOULD be rejected as early as practical.

Business Interpretation

Business language remains authoritative for meaning. Technical teams SHALL demonstrate that domain experts can recognize the represented concept, its owner, its lifecycle, and its exceptions.

A business rule that changes by jurisdiction, agreement, commodity, participant, or time SHALL be modeled as governed policy rather than hidden application logic.

AI Interpretation

AI systems SHALL classify outputs as retrieval, inference, prediction, recommendation, decision, or execution request. Evidence, confidence, authority, and knowledge-time boundaries SHALL remain explicit.

An AI-generated statement does not become a fact, authoritative truth, or decision merely because it is fluent or highly probable.

Architectural Consequences

- Domain boundaries must reflect authority boundaries.
- APIs and events must preserve semantic types and temporal meaning.
- Data lineage must connect projections to authoritative claims.
- Security must distinguish access, authority, delegation, and purpose.
- Operations must preserve evidence during failure and recovery.
- Architecture reviews must demonstrate Canon traceability.

Trade Example

An incorrect container weight is not deleted. A correction references the original assertion, identifies authority and evidence, and becomes effective under explicit temporal rules.

Failure Modes and Risks

- Audit evidence disappears.
- Past decisions become impossible to explain.
- Fraud and operational error are indistinguishable.

Design Rules

1. Model the business concept before choosing storage or messaging technology.
2. Record identity, owner, source, scope, and time.
3. Separate authoritative claims from observations and projections.
4. Preserve corrections and contradictions.
5. Test business invariants at domain boundaries.
6. Require explicit authority before decision or execution.
7. Make conformance observable.

Anti-Patterns

Anti-Pattern	Why It Fails
Database-as-reality	Confuses persistence with existence
Status-field compression	Erases scope, owner, and transition semantics
Silent overwrite	Destroys history
Message-name ambiguity	Confuses request, event, and outcome
AI answer as authority	Collapses knowledge, recommendation, and decision
Shared table ownership	Creates competing truth owners
Implementation-defined policy	Hides business governance

Compliance Requirements

A conforming implementation SHALL provide:

- a model or contract expressing this Canon;
- accountable ownership;
- temporal and provenance metadata;
- testable invariants;
- audit or lineage evidence;
- documented exceptions;
- traceability to affected services, schemas, policies, and runbooks.

Conformance Tests

- ☐ Can prior assertions still be inspected?
- ☐ Can state at a historical time be reconstructed?
- ☐ Are retention and legal-hold rules explicit?

Traceability

Derived Volume	Primary Derivation
Constitution	Authority, ownership, amendment, and conformance
Business Architecture	Capabilities, decision rights, value streams
Domain Architecture	Boundaries, aggregates, commands, events, invariants
AI Architecture	Evidence, uncertainty, authority, decision discipline
Data Architecture	Temporal modeling, lineage, quality, retention
Security Architecture	Identity, delegation, authorization, audit
Engineering	Contract tests and architecture evidence
Operations	Reconstruction, recovery, and incident evidence

Architect's Notes

Architectural Observation

This Canon is valuable only when it changes design behavior. Repeating the statement without encoding ownership, data semantics, policy, and verification is non-conformance.

Rejected Alternatives

- Treating the distinction as documentation-only.
- Allowing every service to redefine the concept.
- Inferring authority from possession of data.
- Removing inconvenient history during correction.
- Deferring semantic validation to analytics.

Related Canons

Dependencies are listed in Classification. Downstream relationships are published in the Traceability Registry.

Review Checklist

- ☐ Does the Canon remain independent?
- ☐ Would removing it make GTOS incoherent?
- ☐ Are definitions non-circular?
- ☐ Are invariants testable?
- ☐ Are examples specific to trade?
- ☐ Are AI and authority consequences explicit?
- ☐ Is implementation traceability available?

Constitutional Amendment Process

Amendment Triggers

An amendment may be proposed when a contradiction, missing authority rule, new trade operating model, regulatory requirement, or proven architecture failure cannot be resolved through lower-level guidance.

Procedure

1. Publish the problem and evidence.
2. Identify affected Canons and Articles.
3. Demonstrate why a lower-level change is insufficient.
4. Analyze compatibility, migration, and governance impact.
5. Run a public or internal review appropriate to classification.
6. Record dissent and rejected alternatives.
7. Approve through the Architecture Council.
8. Release with version, effective date, and transition plan.

Emergency changes MAY be issued temporarily but SHALL expire unless ratified.

Article 01 — Supremacy and Scope

Rule

The Canon governs all normative GTOS architecture. Downstream standards may refine but SHALL NOT contradict it.

Normative Requirements

- The Constitution applies to core platform, extensions, integrations, AI agents, operational procedures, and generated decisions.
- Local implementations MAY add stricter controls.
- A contradiction with Canon is invalid even when technically deployed.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 02 — Truth Ownership

Rule

Every governed proposition SHALL have one accountable truth owner within a declared authority boundary.

Normative Requirements

- Ownership SHALL be published in the domain registry.
- Replication does not transfer ownership.
- Cross-jurisdiction disagreements SHALL be preserved and resolved through explicit authority.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 03 — Identity and Authority

Rule

Identity, authentication, authorization, delegation, mandate, and liability SHALL remain distinct.

Normative Requirements

- Every binding action SHALL identify the acting identity and authority source.
- Service and agent identities SHALL be first-class.
- Emergency authority SHALL be time-bounded and reviewed.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 04 — Decision Governance

Rule

Binding decisions SHALL have authority, evidence, policy, time, and outcome records.

Normative Requirements

- Recommendations SHALL NOT be recorded as decisions.
- High-impact automated decisions require approved policy and evaluation evidence.
- Appeal and correction paths SHALL be defined where decisions affect rights or obligations.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 05 — Historical Integrity

Rule

Governed history SHALL be preserved according to legal, contractual, and operational requirements.

Normative Requirements

- Corrections append new evidence.
- Deletion requests SHALL be reconciled with legal hold and audit obligations.
- Historical reconstruction SHALL be tested for critical domains.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 06 — Interoperability

Rule

GTOS SHALL interoperate through versioned semantic contracts without requiring participants to surrender internal autonomy.

Normative Requirements

- Contracts SHALL declare owner, schema, semantics, compatibility, and lifecycle.
- Canonical semantics do not require one canonical database.
- Adapters SHALL NOT hide loss of material meaning.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 07 — AI Accountability

Rule

AI SHALL operate within delegated authority, declared purpose, evidence constraints, and measurable safety controls.

Normative Requirements

- AI output type SHALL be explicit.
- Tool use SHALL be policy-controlled.
- Model or prompt changes affecting decisions SHALL be traceable and evaluated.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 08 — Security and Privacy

Rule

Security SHALL protect identity, authority, confidentiality, integrity, availability, purpose, and evidence.

Normative Requirements

- Least privilege is mandatory.
- Sensitive data processing SHALL have purpose and jurisdiction context.
- Audit evidence SHALL be protected from unauthorized alteration.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 09 — Conformance and Exceptions

Rule

Conformance SHALL be demonstrated through evidence. Exceptions SHALL be explicit, scoped, owned, time-bounded, and reviewed.

Normative Requirements

- An exception does not amend the standard.
- Repeated exceptions trigger architecture review.
- Critical non-conformance may block release or operation.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Article 10 — Evolution and Amendment

Rule

Semantic evolution SHALL be governed through versioning, migration, and constitutional review.

Normative Requirements

- Canon changes require major-version review.
- Breaking contracts require migration plans.
- Deprecated semantics SHALL have sunset dates and consumers.

Governance Mechanism

The responsible architecture authority SHALL maintain ownership, evidence, exception status, and amendment history for this Article.

Required Evidence

- architecture ownership record;
- applicable policy or registry entry;
- implementation traceability;
- review or approval record;
- conformance test result;
- unresolved exceptions.

Violation Consequence

A material violation SHALL be recorded as architecture debt or operational risk. Where the violation can produce invalid authority, loss of historical truth, security compromise, or harmful automated action, deployment or execution SHALL be blocked until an approved control exists.

Canon Dependencies

All Articles derive from CANON-001 and CANON-002. Additional dependencies SHALL be recorded in the traceability registry.

Volume 01 — GTOS Constitution

The Constitution converts foundational Canons into authority, governance, conformance, and evolution rules.

Constitutional Principle

Architecture is not governed by preference. A normative decision is valid only when its authority, derivation, scope, and evidence are known.

Articles

The ten constitutional articles define supremacy, truth ownership, identity and authority, decisions, historical integrity, interoperability, AI accountability, security, conformance, and evolution.

Volume 02 — Vision

Mission

GTOS enables trustworthy, explainable, interoperable, and resilient coordination of global trade across organizations, jurisdictions, systems, and AI agents.

Strategic Thesis

Global trade is not one workflow or one database. It is a network of independently governed participants that must coordinate shared facts and obligations while retaining distinct authority.

North-Star Properties

- semantic integrity;
- authoritative ownership;
- evidence-aware automation;
- explainable decisions;
- interoperable contracts;
- temporal reconstruction;
- jurisdiction-aware security;
- resilient operations;
- replaceable technology.

Success Measures

GTOS success is measured through lower exception cost, faster evidence acquisition, reduced reconciliation, improved decision quality, shorter recovery time, stronger compliance, and safe automation.

Strategic Principles

1. Model trade reality before software workflow.
2. Optimize for end-to-end evidence, not local data volume.
3. Automate decisions only after authority and knowledge boundaries are explicit.
4. Treat interoperability as semantic governance, not payload transport.
5. Build for jurisdictional variation without fragmenting core meaning.
6. Preserve human accountability while increasing machine capability.
7. Prefer reversible implementation choices and durable business semantics.
8. Design operations and recovery as part of architecture.

Target Operating Model

GTOS operates as a federation of domain authorities connected through governed contracts and platform capabilities.

Participants

- traders and manufacturers;
- logistics providers;
- ports and terminals;
- financial institutions and insurers;
- customs and regulatory authorities;
- inspection and certification bodies;
- platform operators;
- AI agents acting under mandates.

Coordination Pattern

Participants publish authoritative facts within scope, consume perspectives from others, form decisions under policy, express intent, execute through capabilities, and preserve outcomes as evidence.

GTOS does not require universal central ownership. It requires explicit ownership and trustworthy exchange.

Accountability and Decision Rights

RACI Is Insufficient Alone

GTOS additionally requires:

- truth owner;
- policy owner;
- decision authority;
- execution owner;
- evidence custodian;
- risk owner;
- service owner;
- data steward.

Decision-Right Record

Every consequential decision class SHALL define:

- decision name and purpose;
- eligible authority;
- policy basis;
- minimum evidence;
- approval or escalation threshold;
- appeal and correction path;
- expected outcome measures.

Capability 01 — Party and Identity

Purpose

Establish and maintain parties, organizations, persons, facilities, assets, identifiers, credentials, and authority relationships.

Scope

- party onboarding
- identity resolution
- due diligence
- credential lifecycle
- delegation

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 02 — Product and Classification

Purpose

Represent goods, product identities, classifications, attributes, packaging, hazardous characteristics, and jurisdictional codes.

Scope

- product master
- classification
- commodity mapping
- packaging hierarchy
- restricted-goods controls

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 03 — Commercial Agreement

Purpose

Govern offers, contracts, pricing, service levels, Incoterms, obligations, amendments, and expiry.

Scope

- contract lifecycle
- terms
- obligation tracking
- amendment
- performance

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 04 — Order and Fulfilment

Purpose

Manage commercial demand, allocation, commitment, fulfilment plans, and exceptions.

Scope

- order capture
- allocation
- promise
- fulfilment
- change control

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 05 — Shipment and Transport

Purpose

Plan and observe cargo movement, consolidation, booking, custody, milestones, and transport execution.

Scope

- shipment planning
- booking
- routing
- custody
- milestones

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 06 — Trade Documentation

Purpose

Create, validate, exchange, sign, present, amend, and retain trade documents.

Scope

- document assembly
- validation
- signature
- presentation
- retention

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 07 — Compliance and Customs

Purpose

Evaluate controls, declarations, licensing, screening, inspections, release, and evidence.

Scope

- screening
- classification checks
- declaration
- inspection
- release

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 08 — Finance and Insurance

Purpose

Manage payment obligations, settlement, trade finance, guarantees, premiums, claims, and exposure.

Scope

- invoice
- payment intent
- settlement
- finance
- claims

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 09 — Risk and Exception

Purpose

Identify, assess, prioritize, assign, resolve, and learn from risk and exceptions.

Scope

- risk detection
- case creation
- triage
- resolution
- learning

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 10 — Decision Intelligence

Purpose

Acquire evidence, reason, recommend, decide, explain, and monitor outcomes.

Scope

- evidence retrieval
- prediction
- recommendation
- decision
- evaluation

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Capability 11 — Audit and Evidence

Purpose

Preserve provenance, history, decision records, attestations, legal hold, and conformance evidence.

Scope

- lineage
- audit
- attestation
- legal hold
- reconstruction

Required Architecture

The capability SHALL define owner, authoritative propositions, actors, decisions, policies, state, commands, events, measures, service levels, data classifications, and Canon dependencies.

Key Measures

- cycle time;
- first-time-right rate;
- exception rate;
- evidence completeness;
- decision reversal rate;
- automation coverage;
- service reliability.

Risks

The principal failure is local optimization that breaks end-to-end truth, authority, or evidence. Capability reviews SHALL therefore include upstream and downstream value-stream impact.

Volume 03 — Business Architecture

Business Architecture connects the Reference Theory to capabilities, value streams, actors, accountability, and measurable outcomes.

Rules

- Capability ownership SHALL align with truth ownership.
- Value streams SHALL identify decisions and evidence dependencies.
- Business services SHALL publish outcomes and service expectations.
- Automation SHALL preserve decision authority and exception paths.

Value Streams

Trade Formation

Opportunity → qualification → offer → agreement → obligation.

Fulfilment

Order → allocation → shipment plan → booking → movement → delivery.

Border Compliance

Classification → screening → declaration → evidence → inspection or release.

Financial Settlement

Obligation → invoice → payment decision → payment intent → settlement → reconciliation.

Exception Resolution

Detection → qualification → assignment → evidence acquisition → decision → remediation → closure → learning.

Each value stream SHALL identify authoritative facts, decision points, required evidence, hand-offs, service expectations, and failure recovery.

Business Case and Value Model

Cost Baseline

A GTOS business case SHOULD quantify current costs across reconciliation, data correction, document handling, exception management, compliance delay, integration maintenance, duplicated master data, failed automation, fraud exposure, and service disruption. The baseline must use business outcomes rather than only engineering effort.

Value Levers

Value Lever	Mechanism	Typical Evidence
Reduced reconciliation	proposition-level ownership and lineage	fewer manual comparisons and disputes
Faster cycle time	event-driven coordination and decision automation	lower order-to-delivery or clearance time
Lower exception cost	structured cases, evidence requests, and AI assistance	lower mean time to resolution
Safer automation	authority, policy, and Intent controls	fewer unauthorized or duplicated effects
Faster partner onboarding	reusable contracts and identity patterns	reduced integration lead time
Stronger compliance	temporal reconstruction and decision records	faster audit response and fewer control failures
Improved resilience	replay, reconciliation, and recovery design	lower business impact during incidents

Measurement Rules

Benefits SHALL be measured against a dated baseline and attributable scope. Avoid claiming value from automation when the measured improvement results from policy simplification, staffing change, or reduced demand. Leading indicators include contract reuse, evidence completeness, decision latency, and exception age. Lagging indicators include working-capital impact, penalties, service reliability, and dispute cost.

Portfolio Economics

GTOS produces value through reusable foundations. Early domain products may carry a disproportionate share of identity, event, evidence, security, and platform investment. Portfolio funding SHOULD therefore distinguish foundational capabilities from domain-specific delivery.

Value Protection

A local project SHALL NOT bypass shared semantics merely to improve its schedule. Such shortcuts create future reconciliation and migration cost. Architecture debt is recorded with expected economic impact, owner, expiry, and remediation.

Executive Architecture Brief

The GTOS Executive Architecture Brief translates the formal Reference Theory into an actionable transformation model. It explains why GTOS exists, what must be governed centrally, what remains federated, how value is realized, and which risks must be controlled.

Executive Decision

GTOS is not a single application procurement. It is an enterprise operating model implemented through domain products, shared platform capabilities, governed contracts, decision intelligence, security controls, and operational evidence.

Investment Thesis

The principal economic losses in global trade are not caused only by slow software. They are caused by duplicated truth, incompatible identifiers, missing evidence, avoidable exceptions, delayed decisions, manual reconciliation, uncertain authority, and fragile partner integration. GTOS targets these structural costs.

Transformation Outcomes

- authoritative ownership for critical trade propositions;
- reusable domain products rather than project-specific integrations;
- evidence-aware decisions and AI assistance;
- lower exception and reconciliation cost;
- faster partner and jurisdiction onboarding;
- stronger auditability and operational resilience;
- technology replacement without semantic reinvention.

Governance Principle

The executive sponsor owns the transformation outcome. The Architecture Council governs Canon and Constitution. Domain executives own truth and business outcomes. Platform leadership owns shared capabilities. Security, data, AI, and operations provide independent controls and evidence.

Executive Risk Register

Risk	Early Signal	Consequence	Executive Control
GTOS treated as one large application	centralized backlog and database	slow delivery and false ownership	federated domain portfolio
Canon ignored as theory	no traceability in delivery artifacts	semantic drift	mandatory architecture gates
AI given hidden authority	model output directly triggers effects	legal, financial, or safety harm	Decision and Intent controls
Shared platform owns domain truth	platform tables become master	organizational conflict	truth ownership registry
Transformation over-standardizes	jurisdictions and partners cannot fit	adoption failure	extension and policy model
Local teams bypass contracts	direct database and point integration	fragile ecosystem	contract enforcement and deprecation
History is rewritten	corrections update records in place	failed audit and decision review	append-only correction
Security focuses only on roles	mandates and relationships ignored	unauthorized partner or agent access	relationship-aware policy
Recovery restores systems but not business truth	event gaps and duplicates	hidden financial or compliance error	reconciliation exercises

Risk Review

The Architecture Council SHALL review these risks quarterly and at every major release. Each risk has an accountable executive, current exposure, evidence, treatment, residual risk, and next decision date.

Glossary

Aggregate

A domain consistency boundary protecting business invariants.

Authoritative Truth

A governed Claim accepted within a declared authority boundary.

Authority

The recognized right to make a binding assertion, decision, or action.

Canon

A foundational GTOS axiom.

Claim

A proposition asserted to be true.

Command

An authorized request to attempt an action.

Confidence

Qualified support for a Claim; not authority.

Correction

A new assertion that qualifies or supersedes a prior assertion without erasing it.

Decision

A governed commitment or determination.

Delegation

A bounded grant of authority.

Entity

A distinguishable participant, object, place, instrument, document, or concept.

Event

An immutable assertion that a completed occurrence took place.

Evidence

Information used to support, challenge, or audit a Claim or Decision.

Execution

An attempt to realize authorized Intent.

History

The preserved sequence of governed facts, observations, decisions, actions, and corrections.

Identity

Continuity through which an Entity remains distinguishable across change.

Intent

An authorized expression of desired future effect.

Knowledge Set

Claims and evidence available to an observer or process at a time.

Mandate

Authority to pursue a class of Intent under constraints.

Observation

An observer's assertion about reality.

Outcome

The observed result of Execution.

Perspective

A contextual projection or interpretation.

Policy

A governed rule producing permission, obligation, prohibition, or required approval.

Provenance

Origin and transformation history.

Relationship

A business-significant association among independently identified Entities.

State

A scoped set of propositions about a subject at a time.

Truth Owner

The accountable authority governing a proposition.